Security Advisory Digest

AI Components Used

The Security Advisory Digest Agent leverages Artificial Intelligence techniques to automate the collection, classification, and summarization of security advisories.

Primary AI Features

1. **Advisory Ingestion**
    - Collects advisories from multiple trusted sources (vendor feeds, CVE databases, CERT bulletins).
2. **Threat Understanding**
    - Analyzes advisory titles and descriptions.
    - Extracts vulnerability keywords, affected products, and severity context.
3. **Category Classification**
    - Classifies advisories into: · Vulnerability · Patch/Update · Exploit Notice · Other
4. **Severity Prediction**
    - Assigns severity levels: · Critical · High · Medium · Low
5. **Explainable AI**
    - Generates reasoning for every classification.
    - Helps security teams understand why an advisory received a specific category and severity.
6. **ReAct-Based Workflow**
    - Thought: Analyze advisory content.
    - Action: Determine category.
    - Observation: Identify supporting evidence.
    - Action: Determine severity.
    - Observation: Assess potential impact.
    - Final Answer: Generate structured digest output.

AI Models

The system supports:

- OpenAI GPT Models (when API key is available)
- Rule-Based Keyword Classification (fallback mode)

Benefits

- Reduces manual effort in digest preparation.
- Improves awareness and response times.
- Ensures consistent classification.
- Provides transparent decision-making.

Limitations

- Keyword-based mode may miss nuanced context.
- Accuracy depends on advisory quality.
- Requires API access for advanced AI reasoning.

Future AI Enhancements

- Sentiment Analysis of exploit chatter.
- Advisory Similarity Detection.
- Auto Assignment to Security Teams.
- Multi-language Support.
- Learning from Historical Advisories.

Prompt

Act as a senior Python AI engineer.

Build a complete production-quality project called:

SECURITY ADVISORY DIGEST

Goal:
Security teams receive many security advisories daily. The system should automatically ingest advisories, identify which advisories impact the organization's software inventory, generate AI summaries, and produce a report.

Tech Stack:
- Python 3.11
- FastAPI
- SQLite
- Ollama (llama3)
- Feedparser
- Requests
- Pydantic
- Pytest
- Logging

Generate a complete project with the following structure:

security_advisory_digest/

```
├── app.py
├── config.py
├── models.py
│
├── ingestion/
│   ├── __init__.py
│   ├── rss.py
│   └── json_feed.py
│
├── inventory/
│   ├── __init__.py
│   ├── inventory_match.py
│   └── matcher.py
│
```

```
├── llm/
│   ├── __init__.py
│   ├── ollama.py
│   └── llm_service.py
│
├── db/
│   ├── __init__.py
│   └── sqlite.py
│
├── vector/
│   ├── __init__.py
│   └── embeddings.py
│
├── reports/
│   └── report_generator.py
│
├── tests/
│   ├── test_agent.py
│   ├── test_ingestion.py
│   ├── test_inventory_match.py
│   ├── test_database.py
│   └── test_llm.py
│
├── data/
│   ├── inventory.csv
│   └── advisories.db
│
├── prompts.md
├── requirements.txt
└── README.md
```

==========
DATABASE REQUIREMENTS
==========

Create SQLite schema.

Table: advisories

Columns:
- id
- advisory_id
- title
- description
- severity
- vendor
- product
- published_date
- source_feed
- url
- created_at
- updated_at

Table: inventory_items

Columns:
- id
- asset_id
- software_name
- version

Table: matches

Columns:
- id
- advisory_id
- inventory_item_id
- risk_level
- created_at

==========
RSS INGESTION
==========

Read security advisories from:

1. https://github.blog/security/feed/

2. https://www.cisa.gov/cybersecurity-advisories/all.xml

Extract:
- title
- description
- publication date
- URL

Store in SQLite.

==========
INVENTORY MODULE
==========

Load inventory.csv.

Example:

software_name,version
Apache,2.4
Nginx,1.22
OpenSSL,3.0
MySQL,8
Tomcat,10

Match advisories against inventory.

Return affected assets.

==========
OLLAMA INTEGRATION
==========

Connect to local Ollama.

Model:
llama3

Generate:

1. Executive Summary

2. Business Impact

3. Remediation Advice

Prompt Example:

You are a cybersecurity analyst.

Summarize this advisory.

Explain business impact.

Provide remediation steps.

==========
AI AGENT WORKFLOW
==========

Create agent.py.

Workflow:

Step 1
Read RSS feeds

Step 2
Store advisories

Step 3
Load inventory

Step 4
Match advisories

Step 5
Generate AI summary

Step 6
Generate report

Step 7
Return JSON response

Log every step.

==========
FASTAPI API
==========

Create endpoints:

GET /

GET /health

GET /advisories

GET /matches

GET /report

POST /scan

POST /scan should execute the entire agent workflow.

==========
REPORT GENERATION
==========

Generate report.json

Format:

```
{
 "scan_time":"",
 "total_advisories":0,
 "matches":[ ],
 "executive_summary":""
}
```

==========
LOGGING
==========

Implement logging.

Create logs:

INFO
WARNING
ERROR

Log all agent steps.

==========
TESTS
==========

Generate pytest tests for:

RSS ingestion

Inventory matching

SQLite operations

Agent workflow

API endpoints

Happy path only.

==========
README
==========

Generate complete README containing:

Project Overview

Architecture Diagram

Installation

Run Instructions

API Usage

Assumptions

Limitations

Future Improvements

==========
PROMPTS DOCUMENTATION
==========

Generate prompts.md

Include:

Prompt used for summary

Prompt used for remediation

Prompt used for business impact

==========
OUTPUT FORMAT
==========

Generate all source files with complete code.

Do not leave placeholders.

Code must run successfully after:

pip install -r requirements.txt

ollama pull llama3

uvicorn app:app --reload

Generate production-quality code.